Neural Rover: Design and Implementation of a Wi-Fi-Controlled Mobile Robot with Live Pan–Tilt Video

Jubayed Ahmed Sahel, Fahad Hossain Shourov,
Pujon Chakraborty, and Anonna Rahman Spondon
Department of Electrical and Computer Engineering
North South University, Dhaka, Bangladesh
Supervisor: Dr. K. M. A. Salam

***Abstract*—This paper presents the completed CSE499A implementation of *Neural Rover*, a four-wheel mobile robot controlled from a browser over a 2.4-GHz Wi-Fi network. A 30-pin ESP32 hosts a responsive WebSocket interface, drives four DC geared motors through two L298N dual H-bridges, commands a two-axis camera mount, and applies fail-safe stopping. A separate ESP32-CAM captures the scene and supplies a live stream embedded in the same control page. The battery feeds the motor drivers directly while an LM2596 module produces a regulated 5-V logic rail. The system was developed through staged hardware, power, motor, networking, video, and integration tests. Final validation covered translation, steering, held commands, stopping, camera discovery, live video, pan–tilt operation, and reconnection. All hardware construction, firmware integration, safety behavior, testing, documentation, and demonstration tasks specified for CSE499A are complete.**

***Index Terms*—ESP32, ESP32-CAM, mobile robot, WebSocket, L298N, pan–tilt camera, remote control.**

I. Introduction

Mobile robots used for inspection, teaching, and response experiments require both controllable motion and dependable visual feedback. Wireless reliability matters because degraded communication affects the operator's ability to command and observe the platform. NIST has consequently developed repeatable methods for measuring response-robot performance under attenuated radio conditions [1]. Research on TCP/IP mobile-robot teleoperation similarly evaluates throughput and packet jitter as factors that influence controllability [8]. Neural Rover addresses this control-and-observation problem with a compact local system that does not require an Internet cloud service.

The ESP32 was selected as the primary controller because a single device combines 2.4-GHz Wi-Fi, programmable GPIO, multiple UART interfaces, and PWM peripherals [2]. A second ESP32 camera board separates image capture and streaming from time-critical motor control. Espressif's camera driver supports common sensors, frame-buffer management, and conversion paths required for browser streaming [3]. This two-board organization keeps the motor safety loop available while the camera is capturing or encoding a frame.

The purpose of the CSE499A project was to build and verify a rover that can be driven from a phone or laptop, display live video on the controller page, aim the camera using pan and tilt, and enter a safe stopped state whenever control messages disappear. The project-specific firmware and implementation record are maintained by J. A. Sahel in the public CSE499 repository; this report describes the tested configuration represented by the repository history [7].

The eight-week workplan covered component selection, chassis assembly, motor wiring, browser control, camera streaming, pan–tilt integration, communication reliability, complete-system testing, demonstration, and reporting. Every deliverable required for CSE499A has been implemented and checked, giving 100% completion of the planned CSE499A scope [7]. The present paper documents the completed embedded and robotic platform; computer-vision recognition is not part of this report.

II. Purpose and Engineering Requirements

The main objective was intuitive teleoperation with useful visual awareness using inexpensive, accessible modules. The design therefore had to provide directional mobility, prompt response to a held control, video on the same user interface, independent camera aiming, understandable network diagnostics, and a safe response to communication failure. Related mobile-robot work demonstrates that a phone can combine Wi-Fi teleoperation and visual functions in one operator interface [9]. WebSocket was selected here because it establishes a persistent full-duplex TCP connection instead of requiring a new HTTP request for every repeated command [6].

Table I translates the project purpose into acceptance-oriented engineering requirements. The criteria distinguish a feature merely appearing on a webpage from the complete electromechanical behavior required on the physical vehicle. For example, Forward is accepted only when all four wheels agree, while fail-safe behavior is accepted only when the enable outputs return to zero after release, timeout, or loss of connection [7].

The resulting platform is suitable for indoor visual inspection, laboratory teleoperation, embedded-systems education, line-following experiments, and later object-detection studies. These uses depend on the same completed baseline—remote

TABLE I
DESIGN REQUIREMENTS AND ACCEPTANCE CRITERIA

Requirement	Acceptance criterion
Mobility	Four wheels produce correct forward, backward, left, and right motion.
Responsiveness	A held control maintains motion without page navigation or repeated reconnects.
Visual feedback	Live camera frames appear inside the controller page.
View control	Pan and tilt respond independently across a usable mechanical range.
Safety	Release, stale command, browser loss, or Wi-Fi loss stops every motor channel.
Diagnostics	Numeric and fixed local endpoints are reported for controller and camera.
Power integrity	Logic remains operational while motors and servos are commanded.

motion, video feedback, two-axis viewing, local networking, and fail-safe stopping—without changing the fundamental hardware. Wireless robot studies show that communication design is part of robot-system performance rather than a separate convenience [1], [8].

III. METHODOLOGY AND IMPLEMENTATION

A. System Architecture

Fig. 1 separates control, video, actuation, and power paths. The phone or laptop and both ESP32 boards join the same 2.4-GHz `CAR` network. The browser opens the main controller on port 80 and maintains `/ws` for motion and servo messages. The ESP32-CAM serves its diagnostic page on port 80 and its MJPEG stream on port 81. UART2 lets the camera board report its assigned address to the main controller, which then supplies the correct stream host to the webpage [7].

This division assigns clear responsibilities. The main ESP32 owns every actuator and the stop decision. The camera board owns capture, compression, stream delivery, and illumination. The browser is a client rather than a safety controller: if it disappears, the rover stops locally. This arrangement prevents a stalled stream or closed page from directly blocking the motor-control loop [7].

B. Motor, Servo, and UART Interfaces

Two L298N modules provide four H-bridge channels. The L298 is a dual full-bridge driver with separate enable inputs, making each board suitable for two reversible DC motors [4]. The enable jumpers were removed so the main ESP32 can apply PWM to ENA and ENB. Table II gives the final physical assignment. It supersedes early left/right labels: the GPIO32 group is the physical left driver, while the GPIO25 group is the physical right driver. The front-left polarity is corrected in software so that all wheels agree with Forward [7].

The ESP32 LEDC peripheral can generate independent PWM waveforms and supports configurable frequency and duty resolution [2]. Four 12-kHz, 8-bit outputs drive the motor enable pins, while two 50-Hz, 16-bit outputs generate servo pulses. Using different frequency groups permits the motors to receive inaudible high-frequency PWM while the SG90-style servos receive their required low repetition rate. Pan uses GPIO13 and tilt uses GPIO4 of the *main* ESP32 [7].

TABLE II
FINAL MAIN-ESP32 PIN ASSIGNMENT

Subsystem	Signal	GPIO
Left L298N	ENA, IN1, IN2	32, 23, 22
	IN3, IN4, ENB	21, 19, 18
Right L298N	ENA, IN1, IN2	25, 26, 27
	IN3, IN4, ENB	14, 12, 33
Pan–tilt	Pan, tilt	13, 4
UART2	RX2, TX2	16, 17

UART2 runs at 115200 baud. Main RX2 (GPIO16) receives ESP32-CAM U0T, main TX2 (GPIO17) connects to ESP32-CAM U0R, and ground is common. The ESP32 provides multiple configurable UART interfaces for asynchronous data exchange [2]. The camera repeatedly sends a short `CAMIP:` message; line-oriented parsing on the main board validates and stores the address without placing the camera stream itself on the serial link [7].

C. Power Distribution and Electrical Integrity

The 3S battery supplies both L298N motor inputs and the LM2596 input. The LM2596 is a 150-kHz step-down regulator family rated for loads up to 3 A when properly implemented and thermally managed [5]. Its output is adjusted to 5.0 V before connection and supplies the main ESP32, ESP32-CAM, and two servos. Battery negative, regulator negative, both controller grounds, both driver grounds, and servo grounds are joined to establish a common signal reference [7].

Power debugging was performed as an integration activity rather than assuming successful USB testing proved vehicle operation. Motor starting current and switching noise can produce resets, delayed responses, or corrupted serial characters if logic supply or ground is disturbed. The final arrangement separates the battery motor rail from the regulated logic rail, avoids competing USB and buck-converter 5-V sources during normal operation, and uses short, secure power conductors. The wheels are lifted for the first test following any wiring change [4], [5], [7].

The L298N enable jumpers and its 5V-EN jumper have different functions. ENA and ENB jumpers remain removed because the ESP32 supplies PWM. The 5V-EN jumper is configured according to whether the module uses its onboard logic regulator or the external regulated 5-V rail. Treating these jumpers separately prevents accidental connection of two logic supplies [4], [7].

D. Control Firmware and Safety State Machine

The main firmware joins `CAR`, starts an asynchronous web server, and exposes `/ws`. The browser sends `F`, `B`, `L`, or `R` while a button is held, `S` on release, and periodic `K` keepalives. It also sends `P0–P180` and `T0–T180` servo targets. The persistent, full-duplex communication model follows RFC 6455

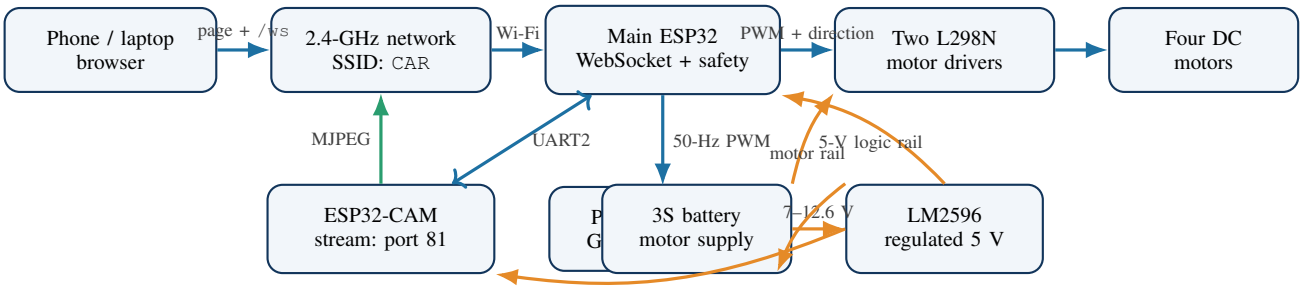


Fig. 1. Implemented Neural Rover architecture and separation of control, video, actuation, and power paths.

and prevents control-button presses from navigating away from the page [6], [7].

Each requested motion passes through a local state machine. PWM begins at duty 120 and rises by 12 every 12 ms toward 255. If direction changes, duty is first set to zero, all direction inputs are neutralized, a 40-ms dead time is observed, and only then is the new polarity applied. If no valid movement or keepalive message arrives within 1300 ms, or the WebSocket or Wi-Fi disconnects, all enable outputs return to zero. The safety outcome therefore does not depend on the browser successfully sending a final Stop packet [7].

The ramp has two functions. First, it overcomes motor starting friction without requesting the full four-motor load in one software step. Second, it makes changes less abrupt while preserving full duty after ramp completion. It is not a closed-loop speed controller because no wheel encoders are installed; wheel speed can still vary with battery voltage, surface load, and motor differences. This distinction avoids claiming precision that the current hardware cannot measure.

E. Camera and Network Integration

The camera firmware first attempts VGA hardware-JPEG capture. The installed sensor reports that JPEG is not supported, so initialization is repeated with RGB565 at QVGA (320×240), one frame buffer, and software JPEG conversion. Espressif notes that RGB/YUV capture creates greater processor and memory load and recommends lower resolutions when JPEG cannot be generated by the sensor [3]. The fallback prioritizes stable streaming over resolution.

Both boards operate as stations on the same hotspot. The main page is available through `rover.local` and the camera through `rovercam.local`; numeric DHCP addresses printed in Serial Monitor remain the fallback. Reconnection logic attempts to restore Wi-Fi without retaining movement. Wireless teleoperation literature emphasizes that delay, packet variation, and loss affect controllability, so the interface exposes connection state and converts loss into a stop rather than hiding it [1], [8].

F. Staged Implementation Procedure

Integration proceeded from low-risk subsystems to the complete vehicle. First, the chassis, motor pairs, and driver outputs were identified without relying on assumed left/right labels. Second, every direction channel was tested with wheels raised and motor polarity was recorded. Third, battery and regulated rails were checked separately. Fourth, the main ESP32 served a minimal movement page before the camera load was introduced. Fifth, the camera stream and UART discovery were verified as independent services. Finally, servos, video, WebSocket control, motor ramping, and fail-safe behavior were exercised together [7].

This order helped distinguish software faults from power faults. A motor that moved in an isolated three-second test demonstrated a functioning output channel, but resets only after connecting a particular channel identified a wiring, driver, motor, or noise issue. Conversely, serial logs showing correct movement commands while the vehicle failed to move directed attention to enable wiring and power rather than the webpage. The final documented configuration records the evidence from these staged checks.

IV. Results, Discussion, and Analysis

A. CSE499A Workplan Completion

Table III maps the eight-week plan to the verified result. Early planning described communication in generic terms; the implemented deliverable is the local WebSocket, UART address discovery, reconnection behavior, and fail-safe control layer used on the physical rover. All CSE499A items are complete [7].

B. Functional Validation

Validation used the final physical wiring and firmware. A movement test began from Stop with wheels lifted, then repeated on the floor after direction agreement was confirmed. A held-command test checked that motion persisted while the pointer remained pressed and stopped upon release. Loss tests closed the browser connection or interrupted message delivery and verified that the watchdog returned the enables to zero. Camera tests checked both the independent camera page and the stream embedded in the controller [7].

Table IV summarizes the acceptance observations. The tests are functional rather than instrumented: they establish correct integrated behavior but do not provide measured latency, radio range, current, or frame rate. Avoiding invented numerical

TABLE III
CSE499A WORKPLAN AND COMPLETION STATUS

Week	Planned focus	Completed evidence	Status
1	Components and circuit plan	Components selected; power, pin, and system diagrams documented	Complete
2	Chassis and motor wiring	Four motors mounted; two L298N modules wired and individually tested	Complete
3	Phone control	Browser motion controls, WebSocket commands, steering, and stop implemented	Complete
4	Camera streaming	ESP32-CAM initialized; camera page and port-81 stream integrated	Complete
5	Pan–tilt mount	Two-axis mount installed and controlled from the main webpage	Complete
6	Communication and safety	UART discovery, keepalive, timeout, disconnect stop, and diagnostics implemented	Complete
7	Integration testing	Motion, video, servos, network recovery, and power behavior tested together	Complete
8	Final demonstration/report	Working system, final firmware, pin document, presentation, and IEEE report prepared	Complete

TABLE IV
INTEGRATED FUNCTIONAL TEST RESULTS

Test	Acceptance observation	Result
Forward/backward	Four wheels agree with command	Pass
Left/right turn	Correct differential direction	Pass
Held command	Motion continues while held	Pass
Release/timeout	Outputs return to stopped state	Pass
Live video	Stream visible in control page	Pass
Camera discovery	UART address reaches main page	Pass
Pan and tilt	Both axes respond to controls	Pass
Reconnect	Control resumes after reconnection	Pass

results is important because teleoperation quality should be evaluated with repeatable measures such as throughput, jitter, and controlled attenuation when quantitative claims are required [1], [8].

C. Motion and User-Interface Analysis

The decisive motor correction was an accurate mapping between the physical sides, driver channels, and software names. Early labels assigned the two driver groups oppositely, and the front-left motor required reversed logical polarity. Correcting these relationships in one software layer produced consistent forward rotation while leaving the already-correct rear motors unchanged. The physical terminal arrangement is now recorded so future maintenance does not reintroduce the error [7].

The WebSocket interface improved usability because held events update the controller over an existing connection. Release sends Stop immediately, while the watchdog remains an independent backup. The page also prevents normal touch interaction from creating browser navigation. This design matches the persistent bidirectional purpose of WebSocket while keeping the actual safety action inside the ESP32 [6].

The controller does not expose a speed slider in the final baseline. Earlier experiments added interface complexity without resolving power and connection faults. The tested version therefore ramps automatically to full duty and prioritizes predictable direction, stop, and connection behavior. A future speed control should be added only after encoder feedback or at least repeatable duty-versus-speed characterization is available.

D. Video, Communication, and Power Analysis

Separating camera and motor workloads proved useful. The main controller remains responsible for movement and safety while the ESP32-CAM performs capture and software JPEG conversion. QVGA reduces memory and processing pressure, but its image detail and frame rate are lower than a compatible hardware-JPEG sensor at VGA. This is an explicit hardware/processing tradeoff rather than webpage cropping [3], [7].

DHCP addresses may change after hotspot reconnection. Combining mDNS names, serially printed numeric endpoints, and UART camera announcements provides three discovery paths. The browser may report connection changes during refresh or Wi-Fi recovery; the firmware interprets those events conservatively and stops motion. Wireless-network quality remains a system-level constraint, as demonstrated by teleoperation research that evaluates packet behavior alongside robot operability [8].

Intermittent resets showed that successful commands over USB did not prove vehicle-level power integrity. Motor current transients and switching noise can disturb the logic rail or common reference. Supplying motor voltage directly from the battery, deriving a regulated 5-V rail with the LM2596, and keeping grounds common produced the intended electrical structure. The L298N still has bipolar-transistor conduction loss, so battery condition, wiring resistance, module temperature, and stall current remain operating limits [4], [5].

V. LIMITATIONS AND EXTENSION POTENTIAL

The present rover has no wheel encoders, current sensor, battery telemetry, or distance sensor. Consequently, it cannot report speed, estimate travelled distance, detect a stalled wheel,

or automatically avoid an obstacle. Open-loop turning also depends on surface friction and motor matching. These limitations do not affect the completed manual-control objectives, but they define the measurements and hardware required for repeatable autonomy.

The camera fallback is suitable for navigation and inspection at short range, yet QVGA software-JPEG streaming limits fine detail and simultaneous viewers. A compatible hardware-JPEG camera would improve resolution and reduce conversion load. Network performance could be characterized later by recording command-to-actuation delay, stream frame rate, packet loss, and useful range under controlled radio attenuation, following the measurement emphasis of existing robot-network studies [1], [8].

The verified platform can support line following with downward-facing reflectance sensors, ultrasonic or time-of-flight obstacle detection, wheel encoders for closed-loop speed, an inertial sensor for motion estimation, or environmental sensing. Such additions should preserve the current stop authority, documented pin ownership, separated power rails, and staged test procedure. This makes the completed CSE499A system a reusable experimental base rather than a single demonstration.

Face recognition is another relevant extension because the ESP32-CAM already provides the video source and the operator device is on the same network. Prior mobile-robot research has combined wireless phone control with off-board face processing [9]. For Neural Rover, a later implementation can send camera frames to a laptop and use an embedding model derived from ArcFace, whose additive angular margin is designed to improve identity separation [10]. This computation is intentionally assigned to the laptop rather than claimed as a completed ESP32 task; enrollment policy, consent, recognition distance, image quality, and false matches would require a separate evaluation.

VI. Conclusion

Neural Rover has reached the complete CSE499A target: a four-wheel vehicle can be driven from a browser, observed through an integrated live camera, and aimed using pan and tilt. The final firmware resolves the physical left/right and front-motor polarity issues, uses persistent commands for smooth operation, and stops safely on release, timeout, browser loss, or network loss. The two-controller architecture keeps motor safety separate from camera processing, while the battery and regulated logic rails provide a clear power structure. All eight weeks of planned CSE499A hardware, firmware, integration, testing, demonstration, and reporting work are complete. The system is ready for later sensing and autonomy work without changing the verified baseline described in this report.

References

[1] K. E. Kimble, R. Candell, and K. S. Saidi, "A test method for measuring response robot performance under attenuated radio conditions," NIST Interagency/Internal Rep. 8243, May 2019, doi: 10.6028/NIST.IR.8243.

[2] Espressif Systems, "ESP32 series datasheet," ver. 5.2, 2025. [Online]. Available: https://documentation.espressif.com/esp32_datasheet_en.pdf

[3] Espressif Systems, "ESP32 camera driver," *esp32-camera*. [Online]. Available: https://github.com/espressif/esp32-camera

[4] STMicroelectronics, "L298 dual full-bridge driver," datasheet. [Online]. Available: https://www.st.com/resource/en/datasheet/l298.pdf

[5] Texas Instruments, "LM2596 SIMPLE SWITCHER power converter 150-kHz 3-A step-down voltage regulator," datasheet. [Online]. Available: https://www.ti.com/lit/ds/symlink/lm2596.pdf

[6] I. Fette and A. Melnikov, "The WebSocket protocol," RFC 6455, Dec. 2011, doi: 10.17487/RFC6455.

[7] J. A. Sahel, "CSE499 Neural Rover project source code and documentation," GitHub repository, 2026. [Online]. Available: https://github.com/Jubayed-Sahel/CSE499

[8] K. Sawai, S. Aoyama, T. Motoyoshi, T. Oshima, K. Koyanagi, H. Masuta, and T. Tamamoto, "A mobile robot teleoperation system with a wireless communication infrastructure using a leaky coaxial cable based on TCP/IP," *Int. J. Adv. Comput. Sci. Appl.*, vol. 10, no. 7, 2019, doi: 10.14569/IJACSA.2019.0100713.

[9] P. Papcun, I. Zolotova, and K. Tafsi, "Control and teleoperation of robot Khepera via Android mobile device through Bluetooth and WiFi," *IFAC-PapersOnLine*, vol. 49, no. 25, pp. 188–193, 2016, doi: 10.1016/j.ifacol.2016.12.032.

[10] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive angular margin loss for deep face recognition," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, 2019, pp. 4690–4699, doi: 10.1109/CVPR.2019.00482.